Tarea para PSP04

Actividad 4.1. Modifica el ejercicio 1 de la unidad 3 para el servidor permita trabajar de forma concurrente con varios clientes.

CÓDIGO SERVIDOR

```
package actividad41;


import java.io.*;

import java.net.*;


public class ServidorAdivina extends Thread {


    // Para guardar el socket de cada cliente específico y la cuenta
global

    Socket skCliente;

    static int contadorGlobal = 1;

    //Variable para identificar a cada cliente

    int idCliente;

    static final int PUERTO = 2000;





    // Constructor: Recibe el socket del cliente
```

```
    public ServidorAdivina(Socket sCliente ,int id) {

        this.skCliente = sCliente;

        this.idCliente=id;

    }



    // Cambiamos el MAIN: ahora se encarga solo de aceptar conexiones
y lanzar hilos

    public static void main(String[] args) {

        try {

            ServerSocket skServidor = new ServerSocket(PUERTO);

            System.out.println("Servidor Adivina (Concurrente)
escuchando en el puerto " + PUERTO);



            while (true) {

                // Se conecta un cliente

                Socket sCliente = skServidor.accept();




                // Creamos una nueva instancia de esta clase (Thread)
y la iniciamos

                new ServidorAdivina(sCliente,contadorGlobal).start();

                System.out.println("Nuevo cliente "+ contadorGlobal +
" conectado para jugar.");

                contadorGlobal++;
```

```
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // El RUN contiene la lógica que ejecuta el run para ese cliente
    @Override
    public void run() {
        try {
            //Flujos
            DataInputStream flujo_entrada = new
DataInputStream(skCliente.getInputStream());
            DataOutputStream flujo_salida = new
DataOutputStream(skCliente.getOutputStream());

            //Generar número secreto (0 al 100)
            int numeroSecreto = (int) (Math.random() * 101);
            boolean juegoTerminado = false;

            while (!juegoTerminado) {
                //Leer intento del cliente
                int numeroCliente = flujo_entrada.readInt();
```

```
                if (numeroCliente == numeroSecreto) {

                    flujo_salida.writeUTF("Acertaste");

                    juegoTerminado = true;

                } else if (numeroCliente < numeroSecreto) {

                    flujo_salida.writeUTF("Mayor");

                } else {

                    flujo_salida.writeUTF("Menor");

                }

            }

            //No cierro skServidor.close() como en el secuencial
porque tenemos un ciclo sin fin.

            skCliente.close();

            System.out.println("El cliente " + this.idCliente + " ha
acertado. Cerrando servidor.");




        } catch (Exception e) {

            System.out.println("Error en el thread del cliente " +
this.idCliente + ": " + e.getMessage());

        }

    }

}
```

CÓDIGO CLIENTE

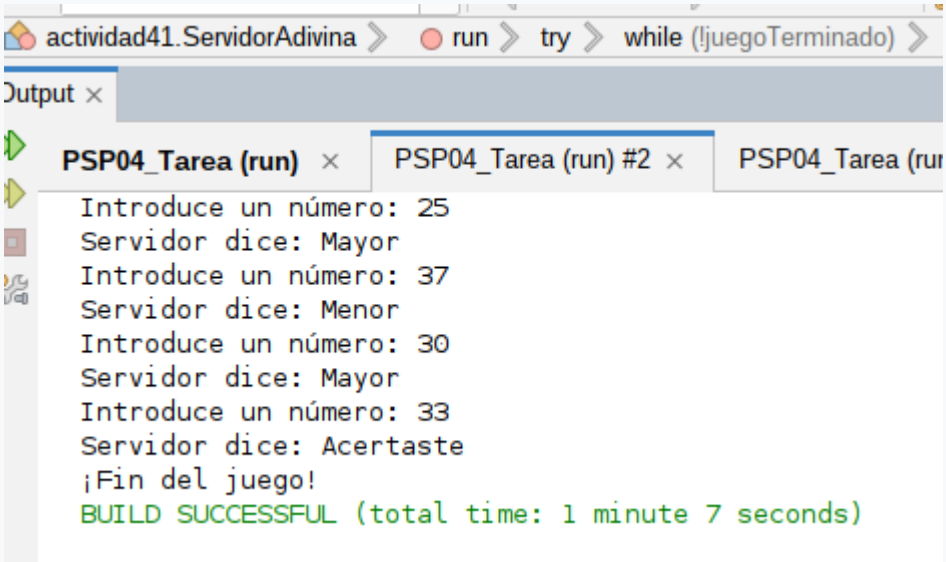
El cliente se queda igual que en el de la anterior unidad

```
package actividad41;


import java.io.*;

import java.net.*;

import java.util.Scanner;


public class ClienteAdivina {

    static final String HOST = "localhost";

    static final int PUERTO = 2000;


    public static void main(String[] args) {

        try {

            Socket sCliente = new Socket(HOST, PUERTO);

            System.out.println("CONECTADO: Intenta adivinar el número
secreto (0-100)");


            // Flujos de datos

            DataOutputStream flujo_salida = new
DataOutputStream(sCliente.getOutputStream());
```

```
            DataInputStream flujo_entrada = new
DataInputStream(sCliente.getInputStream());


            Scanner sc = new Scanner(System.in);

            boolean acertado = false;


            while (!acertado) {

                System.out.print("Introduce un número: ");

                int n = sc.nextInt();


                // Enviar número

                flujo_salida.writeInt(n);


                // Recibir respuesta

                String respuesta = flujo_entrada.readUTF();

                System.out.println("Servidor dice: " + respuesta);


                if (respuesta.equals("Acertaste")) {

                    acertado = true;

                    System.out.println("¡Fin del juego!");

                }

            }
```

```
            sCliente.close();


        } catch (Exception e) {

            System.out.println("Error: " + e.getMessage());

        }

    }

}
```

Probamos el código: Cliente 1



Cliente 2

```
Find:                Previous   Next   Select
actividad41.ServidorAdivina > run > try > while (!juegoTerminado) > if (numeroCliente == r
Output ×
PSP04_Tarea (run) ×   PSP04_Tarea (run) #2 ×   PSP04_Tarea (run) #3 ×
Introduce un número: 65
Servidor dice: Mayor
Introduce un número: 70
Servidor dice: Mayor
Introduce un número: 73
Servidor dice: Mayor
Introduce un número: 74
Servidor dice: Acertaste
¡Fin del juego!
BUILD SUCCESSFUL (total time: 1 minute 35 seconds)
```

Servidor

```
Find:                Previous   Next   Select
actividad41.ServidorAdivina > run > try > while (!juegoTerminado) > if (numeroClien
Output ×
PSP04_Tarea (run) ×   PSP04_Tarea (run) #2 ×   PSP04_Tarea (run) #3 ×
run:
Servidor Adivina (Concurrente) escuchando en el puerto 2000
Nuevo cliente 1 conectado para jugar.
Nuevo cliente 2 conectado para jugar.
El cliente1 ha acertado. Cerrando servidor.
El cliente2 ha acertado. Cerrando servidor.
```

Actividad 4.2. Modifica el ejercicio 2 de la unidad 3 para el servidor permita trabajar de forma concurrente con varios clientes.

CÓDIGO SERVIDOR

```
package actividad42;



import java.io.*;
```

```
import java.net.*;


public class ServidorFichero extends Thread {


    Socket skCliente;

    static final int PUERTO = 1500;

    //Igual que en la actividad 41, he añadido contadores de cliente

    static int contadorGlobal = 0;

    int idCliente;


    public ServidorFichero(Socket sCliente, int id) {

        this.skCliente = sCliente;

        this.idCliente = id;

    }


    public static void main(String[] args) {

        try {

            ServerSocket skServidor = new ServerSocket(PUERTO);

            System.out.println("Servidor de Ficheros (Concurrente)
listo en puerto " + PUERTO);


            while (true) {

                Socket sCliente = skServidor.accept();
```

```
                contadorGlobal++;

                System.out.println("Cliente " + contadorGlobal + "
conectado.");



                // Lanzamos el thread con el socket y el ID

                new ServidorFichero(sCliente,
contadorGlobal).start();

            }

        } catch (Exception e) {

            System.out.println(e.getMessage());

        }

    }



    @Override

    public void run() {

        try {

            DataInputStream flujo_entrada = new
DataInputStream(skCliente.getInputStream());

            DataOutputStream flujo_salida = new
DataOutputStream(skCliente.getOutputStream());



            boolean ficheroEnviado = false;
```

```
            // BUCLE: Repetimos mientras NO hayamos enviado el
fichero

            while (!ficheroEnviado) {

                // 1. Recibir nombre del fichero

                String nombreFichero = flujo_entrada.readUTF();

                File fichero = new File(nombreFichero);


                // 2. Comprobar existencia

                if (fichero.exists() && fichero.isFile()) {

                    flujo_salida.writeUTF("OK");  // Avisamos al
cliente que existe


                    // Leemos y enviamos el fichero

                    BufferedReader lector = new BufferedReader(new
FileReader(fichero));

                    String linea;

                    while ((linea = lector.readLine()) != null) {

                        flujo_salida.writeUTF(linea);

                    }

                    lector.close();


                    // Señal de fin de fichero

                    flujo_salida.writeUTF("EOF");
```

```
                    // Marcamos como enviado para salir del bucle

                    ficheroEnviado = true;

                    System.out.println("Cliente " + idCliente + ": 
Fichero enviado exitosamente.");



                } else {

                    flujo_salida.writeUTF("ERROR");

                    // Volvemos al inicio del bucle a esperar otro 
nombre

                }

            }



            // No cerramos skServidor

            skCliente.close();

            System.out.println("Cliente " + idCliente + " 
desconectado.");



        } catch (Exception e) {

            System.out.println("Error o desconexión en Cliente " + 
idCliente + ": " + e.getMessage());

        }

    }

}
```

CÓDIGO CLIENTE

El cliente se queda igual que en el de la anterior unidad

```
package actividad42;


import java.io.*;

import java.net.*;

import java.util.Scanner;


public class ClienteFichero {

    static final String HOST = "localhost";

    static final int PUERTO = 1500;


    public static void main(String[] args) {

        try {

            Scanner sc = new Scanner(System.in);

            Socket sCliente = new Socket(HOST, PUERTO);


            DataOutputStream flujo_salida = new
DataOutputStream(sCliente.getOutputStream());

            DataInputStream flujo_entrada = new
DataInputStream(sCliente.getInputStream());


            boolean ficheroRecibido = false;
```

```
           // Repetimos hasta recibir el fichero correctamente

            while (!ficheroRecibido) {

                System.out.print("Introduce el nombre del fichero que
quieres ver: ");

                String ficheroSolicitado = sc.nextLine();


                flujo_salida.writeUTF(ficheroSolicitado);


                String estado = flujo_entrada.readUTF();

                if (estado.equals("OK")) {

                    System.out.println("--- Contenido del Fichero ---
");


                    String linea = flujo_entrada.readUTF();

                    while (!linea.equals("EOF")) {

                        System.out.println(linea);

                        linea = flujo_entrada.readUTF();

                    }

                    System.out.println("----------------------------
");

                    System.out.println("Fichero recibido. Finalizando
conexión.");

                    ficheroRecibido = true; // Salimos del bucle
```

```
                }else if (estado.equals("ERROR")) {

                    System.out.println("Error: El fichero no existe.
Inténtalo de nuevo.\n");

                }

            }



            sCliente.close();



        } catch (Exception e) {

            System.out.println("Error: " + e.getMessage());

        }

    }

}
```

Así se ve el servidor cuando se conectan diferentes instancias.

```
PSP04_Tarea (run) ×    PSP04_Tarea (run) #2 ×
Servidor de Ficheros (Concurrente) escuchando en puerto 1500
Cliente 1 conectado.
Cliente 1: Fichero enviado exitosamente.
Cliente 1 desconectado.
Cliente 2 conectado.
Cliente 2: Fichero enviado exitosamente.
Cliente 2 desconectado.
Cliente 3 conectado.
Cliente 3: Fichero enviado exitosamente.
Cliente 3 desconectado.
```

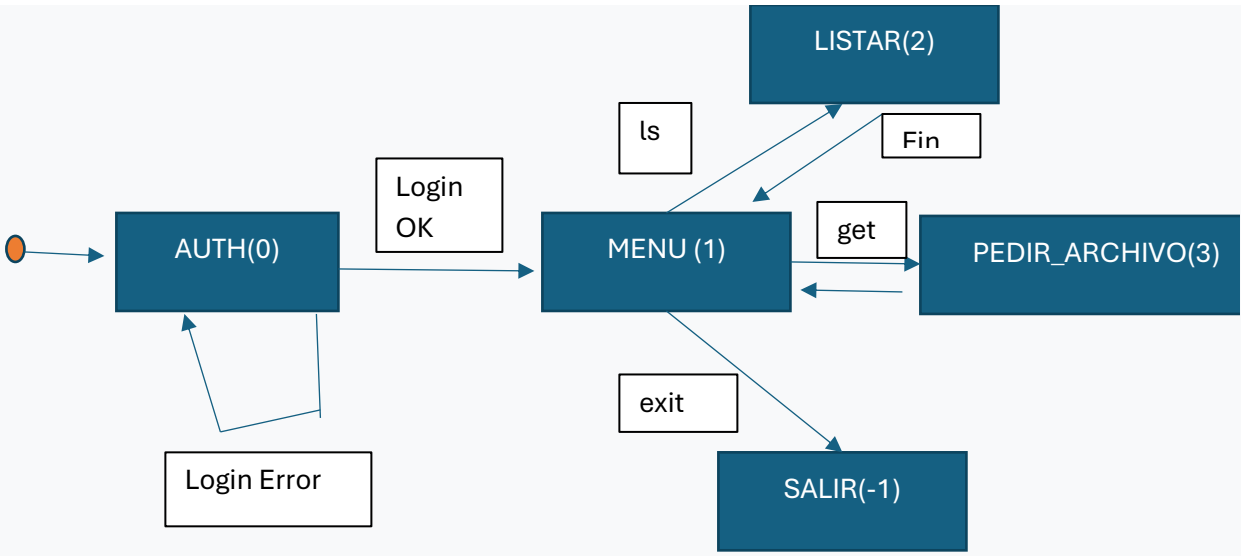
Y aquí tenemos un ejemplo de cliente.

```
actividad42.ClienteFichero
Output
PSP04_Tarea (run)    PSP04_Tarea (run) #2
run:
Introduce el nombre del fichero que quieres ver: hola.txt
Error: El fichero no existe. Inténtalo de nuevo.

Introduce el nombre del fichero que quieres ver: prueba.txt
--- Contenido del Fichero ---
¡Hola desde prueba!
-----------------------------
Fichero recibido. Finalizando conexión.
BUILD SUCCESSFUL (total time: 6 seconds)
```

Actividad 4.3. A partir del ejercicio anterior crea un servidor que una vez iniciada sesión a través de un nombre de usuario y contraseña específico (por ejemplo javier / secreta) el sistema permita Ver el contenido del directorio actual, mostrar el contenido de un determinado archivo y salir.

Para realizar el ejercicio primero debes crear un diagrama de estados que muestre el funcionamiento del servidor.

- **Inicio** -> Estado **AUTH(0)** (Espera usuario-pass).
- Si login OK -> Transición a Estado **MENU(1)**.
- Si no, seguimos en AUTH.
- Estado **MENU(1)** -> Espera comando:
- ls -> Va a Estado **LISTAR(2)** -> Muestra y vuelve a **MENU(1)**.
- get -> Va a Estado **PEDIR_ARCHIVO(3)** -> Pide nombre, muestra y vuelve a **MENU(1)**.
- exit -> Va a Estado **SALIR(-1)** (Fin).



CÓDIGO SERVIDOR

```
package actividad43;


import java.io.*;

import java.net.*;


public class ServidorEstados extends Thread {


    Socket skCliente;

    static final int PUERTO = 2500;


    // Constantes para los estados

    final int ESTADO_AUTH = 0;

    final int ESTADO_MENU = 1;

    final int ESTADO_LISTAR = 2;
```

```
    final int ESTADO_PEDIR_ARCHIVO = 3;

    final int ESTADO_SALIR = -1;


    public ServidorEstados(Socket sCliente) {

        this.skCliente = sCliente;

    }


    public static void main(String[] args) {

        try {

            ServerSocket skServidor = new ServerSocket(PUERTO);

            System.out.println("Servidor de Estados escuchando en
puerto " + PUERTO);


            while (true) {

                Socket sCliente = skServidor.accept();

                System.out.println("Nuevo cliente conectado.");

                // Lanzamos el thread para atender a este cliente en
particular

                new ServidorEstados(sCliente).start();

            }

        } catch (Exception e) {

            System.out.println(e.getMessage());

        }
```

```
    }



    @Override

    public void run() {

        try {

            DataInputStream flujo_entrada = new
DataInputStream(skCliente.getInputStream());

            DataOutputStream flujo_salida = new
DataOutputStream(skCliente.getOutputStream());



            int estado = ESTADO_AUTH; //Inicializamos en el estado 0

            boolean salir = false;



            //Típico menú de java en consola, un switch dentro de un
while

            while (!salir) {

                switch (estado) {

                    case ESTADO_AUTH:

                        flujo_salida.writeUTF("LOGIN: Introduce
'usuario-contraseña' (ej: javier-secreta):");

                        String credenciales =
flujo_entrada.readUTF();

                        if (credenciales.equals("javier-secreta")) {

                            flujo_salida.writeUTF("OK: Acceso
permitido.");
```

```
                            estado = ESTADO_MENU;

                        } else {

                            flujo_salida.writeUTF("ERROR:
Credenciales incorrectas.");

                            // Nos quedamos en ESTADO_AUTH

                        }

                        break;



                    case ESTADO_MENU: // Estado 1

                        flujo_salida.writeUTF("MENU: Introduce
comando (ls / get / exit):");

                        String comando = flujo_entrada.readUTF();



                        if (comando.equals("ls")) estado =
ESTADO_LISTAR;

                        else if (comando.equals("get")) estado =
ESTADO_PEDIR_ARCHIVO;

                        else if (comando.equals("exit")) estado =
ESTADO_SALIR;

                        else flujo_salida.writeUTF("ERROR: Comando no
reconocido.");

                        break;



                    case ESTADO_LISTAR: // Estado 2
```

```
                        File dir = new File("."); // Directorio
actual (raíz del proyecto)

                        File[] archivos = dir.listFiles();

                        String lista = "--- LISTADO DE FICHEROS ---
\n";

                        if (archivos != null) {

                            for (File f : archivos) {

                                if (f.isFile()) lista += f.getName()
+ "\n";

                            }

                        }

                        lista += "----------------------------";

                        flujo_salida.writeUTF(lista);

                        estado = ESTADO_MENU;

                        break;


                    case ESTADO_PEDIR_ARCHIVO: // Estado 3

                        flujo_salida.writeUTF("ARCHIVO: Introduce el
nombre del fichero:");

                        String nombre = flujo_entrada.readUTF();

                        File f = new File(nombre);


                        if (f.exists() && f.isFile()) {
```

```
                            // Leemos el fichero y lo guardamos en un
String para mandarlo al cliente

                            StringBuilder contenido = new
StringBuilder("--- CONTENIDO ---\n");

                            BufferedReader br = new
BufferedReader(new FileReader(f));

                            String linea;

                            while ((linea = br.readLine()) != null) {

                                contenido.append(linea).append("\n");

                            }

                            br.close();

                            contenido.append("-----------------");


flujo_salida.writeUTF(contenido.toString());

                        } else {

                            flujo_salida.writeUTF("ERROR: El fichero
no existe.");

                        }

                        estado = ESTADO_MENU;

                        break;



                    case ESTADO_SALIR: // Estado -1

                        flujo_salida.writeUTF("BYE: Cerrando
conexión.");

                        salir = true;
```

```
                        break;

                }

            }

            skCliente.close();

            System.out.println("Cliente desconectado.");


        } catch (Exception e) {

            System.out.println("Error en comunicación: " +
e.getMessage());

        }

    }

}
```

CÓDIGO CLIENTE

```
package actividad43;


import java.io.*;

import java.net.*;

import java.util.Scanner;


public class ClienteEstados {

    static final String HOST = "localhost";
```

```
    static final int PUERTO = 2500;



    public static void main(String[] args) {

        try {

            Socket sCliente = new Socket(HOST, PUERTO);

            DataOutputStream flujo_salida = new
DataOutputStream(sCliente.getOutputStream());

            DataInputStream flujo_entrada = new
DataInputStream(sCliente.getInputStream());

            Scanner sc = new Scanner(System.in);



            System.out.println("Conectado al servidor. Esperando
instrucciones...");

            boolean conectado = true;



            while (conectado) {

                // Mensaje del servidor

                String mensaje = flujo_entrada.readUTF();

                System.out.println("[SERVIDOR] " + mensaje);



                // Comprobamos si hay que responder o si nos
desconectan con BYE

                if (mensaje.startsWith("BYE")) {

                    conectado = false;
```

```
                }
                else if (mensaje.startsWith("LOGIN") ||
                         mensaje.startsWith("MENU") ||
                         mensaje.startsWith("ARCHIVO")) {
                    // Si el servidor pide datos, escribimos
                    System.out.print("> ");
                    String respuesta = sc.nextLine();
                    flujo_salida.writeUTF(respuesta);
                }
                // Si el mensaje es "OK", "ERROR" o el listado de
ficheros,
                // simplemente se muestra
            }

            sCliente.close();
            System.out.println("Fin de la sesión.");

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

PRUEBAS

De parte del servidor

```
PSP04_Tarea (run) ×   PSP04_Tarea (run) #2 ×
run:
Servidor de Estados escuchando en puerto 2500
Nuevo cliente conectado.
Cliente desconectado.
```

De parte del cliente

```
PSP04_Tarea (run) ×   PSP04_Tarea (run) #2 ×
run:
Conectado al servidor. Esperando instrucciones...
[SERVIDOR] LOGIN: Introduce 'usuario-contraseña' (ej: javier-secreta):
> rodrigo-secreta
[SERVIDOR] ERROR: Credenciales incorrectas.
[SERVIDOR] LOGIN: Introduce 'usuario-contraseña' (ej: javier-secreta):
> javier-secreta
[SERVIDOR] OK: Acceso permitido.
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> ls
[SERVIDOR] --- LISTADO DE FICHEROS ---
build.xml
manifest.mf
prueba.txt
---------------------------
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> get
[SERVIDOR] ARCHIVO: Introduce el nombre del fichero:
> prueba.txt
[SERVIDOR] --- CONTENIDO ---
¡Hola desde prueba!
----------------
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> exit
[SERVIDOR] BYE: Cerrando conexión.
Fin de la sesión.
BUILD SUCCESSFUL (total time: 46 seconds)
```

Más pruebas de cliente

PSP04_Tarea (run) × | PSP04_Tarea (run) #2 ×

```
run:
Conectado al servidor. Esperando instrucciones...
[SERVIDOR] LOGIN: Introduce 'usuario-contraseña' (ej: javier-secreta):
> javier-secreta
[SERVIDOR] OK: Acceso permitido.
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> get
[SERVIDOR] ARCHIVO: Introduce el nombre del fichero:
> hola.txt
[SERVIDOR] ERROR: El fichero no existe.
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> manifest.mf
[SERVIDOR] ERROR: Comando no reconocido.
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> prueba.txt
[SERVIDOR] ERROR: Comando no reconocido.
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> get
[SERVIDOR] ARCHIVO: Introduce el nombre del fichero:
> prueba.txt
[SERVIDOR] --- CONTENIDO ---
¡Hola desde prueba!
----------------
[SERVIDOR] MENU: Introduce comando (ls / get / exit):
> exit
[SERVIDOR] BYE: Cerrando conexión.
Fin de la sesión.
BUILD SUCCESSFUL (total time: 57 seconds)
```